

Case Study: File Systems in Windows and Apple Platforms

Course Capstone — The Final Lecture

Every file-system idea this course has built up — files as a named abstraction over disk blocks, tree-structured directories, owner/group/other permissions, journaling for crash safety — is, almost unchanged, how the file system on the computer you are reading this on actually works. This lecture looks at two real, production file systems side by side, then closes the entire course.

In This Lecture

- NTFS basics: the Master File Table, journaling for crash consistency, and ACL-based permissions
- APFS basics: copy-on-write, space sharing across volumes, and native snapshots and encryption
- A direct, dimension-by-dimension comparison of NTFS and APFS
- A complete course recap — all thirty-two lectures, seven units, one running example followed end to end
- Where the ideas in this course lead next, and a closing note now that the course is complete

Windows File System: NTFS — The Master File Table

NTFS organizes almost everything around a single database-like structure called the Master File Table (MFT). Every file on an NTFS volume — and every piece of file-system metadata too — is represented as one record in this table.

MFT Record

Holds a file's name, timestamps, security descriptor, and pointers to its data clusters



Small file: stored entirely within its own record

A file small enough to fit in the record's remaining space needs no separate data clusters at all — it is “resident” data

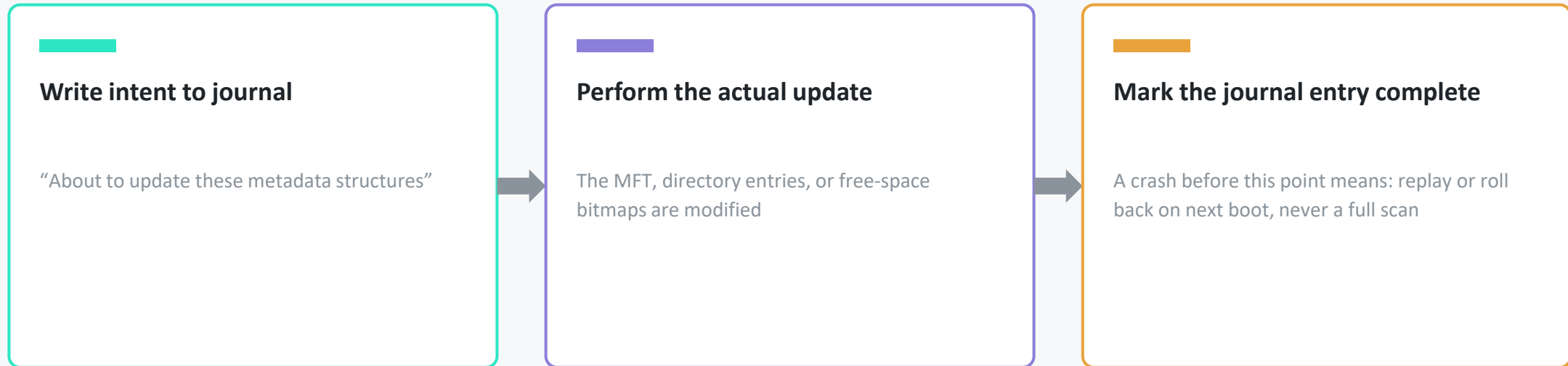


Large file: record points out to data clusters

The record holds only the map; the actual bytes live in clusters elsewhere on the volume

Journaling: Log the Intent, Then Act

NTFS is a journaling file system: before it modifies its own metadata structures, it first writes a short record of the change it is about to make into a separate log area, the journal, and only afterward performs the actual modification.



NTFS Permissions via ACLs

NTFS enforces file protection with full access control lists (ACLs) attached to every file and directory — exactly the access-list protection model Lecture 31 introduced as the flexible-but-potentially-unwieldy alternative to owner/group/other.

- Each entry in an NTFS ACL names a specific user or group and the specific operations they are permitted (or explicitly denied)
- This is considerably more expressive than three fixed rwx buckets — a single file can grant five different users five different permission sets
- The cost is exactly the administrative overhead Lecture 31 warned a growing access list accumulates: every new user who needs access means another entry to add, audit, and eventually remove

Apple File System: APFS

APFS is the modern default file system across macOS, iOS, iPadOS, tvOS, and watchOS, replacing the older HFS+. It was designed from scratch around flash storage and makes different trade-offs from NTFS at almost every point.

- Copy-on-write (COW) is the foundation — the same core idea Lecture 25's `fork()` discussion introduced for duplicating a process's memory, now applied to an entire volume's data
- When a file (or an entire snapshot) is “copied,” APFS does not duplicate its data blocks up front — the copy and the original share the same underlying blocks
- A genuinely new block is allocated only the moment either side is actually modified — the identical lazy-duplication principle, now protecting file data instead of process memory pages

APFS: Space Sharing Across Volumes

Older partitioning schemes gave each volume a fixed size, decided at creation time — growing one volume meant shrinking another, often requiring a disruptive repartition.

Older partitioning

Each volume's size is fixed at creation; growing one means shrinking another

APFS container

Multiple volumes share one pool of free space dynamically; no volume's size is rigidly fixed

APFS: Snapshots and Encryption

Because of its copy-on-write foundation, APFS can take a snapshot — a complete, point-in-time, read-only copy of an entire volume — almost instantly and at almost no extra disk cost.

- The snapshot simply shares every block with the live volume at the moment it was taken, and only consumes additional space as the live volume's data subsequently diverges from it, block by block
- This is precisely the mechanism behind Time Machine-style backups on macOS, which can hold many historical snapshots without storing a full, separate copy of the data for each one
- APFS also natively supports strong, full-volume and per-file encryption, built in as a first-class feature rather than bolted on afterward

NTFS and APFS, Dimension by Dimension

Both file systems solve the exact problems the last three lectures raised — just with different concrete engineering choices.

Dimension	NTFS	APFS
Journaling approach	Logs metadata-change intent before acting; replays/rolls back the journal after a crash — no full-disk check	Copy-on-write means an in-progress update never overwrites committed data in place, so there is little to “replay” at all
Space allocation model	Each volume has a size set when the partition is created	Multiple volumes dynamically share free space from one physical container
Snapshot support	Supported (Volume Shadow Copy), layered on top of the base file system rather than native to it	Native, instant, and space-efficient by design, directly because of its copy-on-write foundation
Typical use context	Windows desktops, laptops, and servers; fine-grained ACL permissions for shared, multi-user environments	Apple desktops and mobile devices; flash-optimized, snapshot-friendly, built-in encryption by default

NTFS and APFS: Two Answers to the Same Problems

- NTFS's ACLs and APFS's owner/group/other-plus-extensions both answer Lecture 31's protection question, from two different directions
- NTFS's journal and APFS's copy-on-write both answer the exact same crash-consistency problem, just differently

Different defaults, same underlying problems

Neither design is “wrong” — they reflect different priorities (fine-grained enterprise permission management vs. flash-optimized, snapshot-native simplicity) built on top of the exact same concepts this course spent four lectures establishing.

Course Recap — Seven Units, One Running Example

1

One running example, threaded through all 7 units

2

Unit 1: OS Foundations and Structure

3

Unit 2: Process Management and Threads

4

Unit 3: CPU Scheduling

5

Unit 4: Process Synchronization

6

Unit 5: Deadlocks

7

Unit 6: Memory Management

8

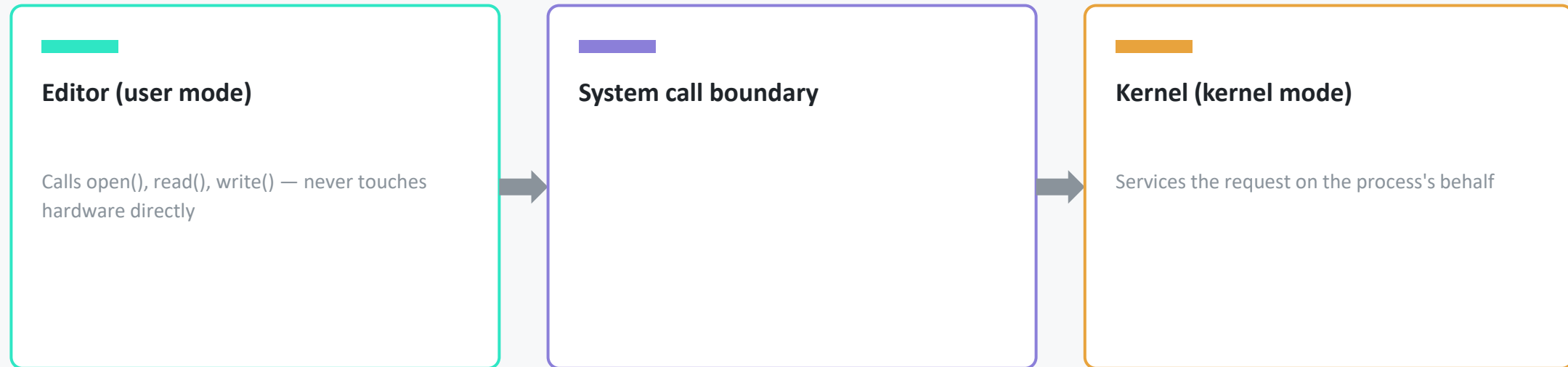
Unit 7: Storage and File Systems

The Running Example

Throughout this recap, one scenario recurs: Ayesha opens a text editor to edit and save a large report. It is small enough to hold in your head in full, and rich enough that every unit of this course had something genuine to say about it.

Unit 1 — OS Foundations and Structure

The moment Ayesha launches the editor, the OS creates a new process for it, and every single thing that process does from then on — opening the report, allocating memory, eventually saving — happens through a system call, the formal, protected boundary between an ordinary program and the kernel.



Process Management and Threads

The editor doesn't do everything on its single main thread — it spins up a background autosave thread that periodically saves the report without freezing the editing window. Both threads share the same process's address space and open files, exactly the distinguishing feature of threads: cheaper to create and switch between than a whole new process, precisely because so much context is already shared.

CPU Scheduling

Ayesha's editor is never the only thing running — a browser, a music player, and a dozen background services are all runnable at the same moment. The CPU scheduler decides, many times a second, which of them actually gets the processor next, keeping every process, including the editor's own main and autosave threads, making visible progress without any one of them starving the others.

Process Synchronization

The main thread and the autosave thread both need to touch the exact same in-memory copy of the report. Without coordination, the autosave thread could read the buffer mid-edit and save a half-typed sentence — or the two threads could corrupt the buffer entirely. A lock around every access to the shared buffer is what prevents that race, exactly the critical-section discipline this unit built from first principles.

Deadlocks

Suppose the autosave thread acquires the buffer lock first and then needs the log-file lock, while at the same moment the main thread acquires the log lock first and then needs the buffer lock. Neither thread can proceed, and neither will ever release what it's holding — a deadlock. The fix is just as direct: enforce a single, consistent order in which any thread acquires these two locks, and the circular wait deadlock depends on becomes structurally impossible.

Unit 6 — Memory Management

The report's text, and the editor's own code, do not all sit in physical RAM the instant the editor launches — they are demand-paged in, one page at a time, only as each page is actually touched. If Ayesha opens a second window on the same report, the OS does not duplicate every page immediately; it uses copy-on-write to let both windows share the same physical pages until one of them actually modifies its own copy.

Page first touched

Page fault — the OS brings it into a physical frame on demand



Second window opened on the same report

Copy-on-write: both windows share the same physical pages, unmodified



Either window edits its own copy

Only now does a genuinely separate physical page get allocated

Storage and File Systems

Finally, Ayesha presses “Save.” That one keystroke becomes a `write()` system call that descends through the layered file-system implementation — the logical file system, the file-organization module, the basic file system, and I/O control — checked the entire way against the report file's `rw`x permissions, and committed to disk through an NTFS journal or APFS copy-on-write mechanism — guaranteeing that even a crash one instant later cannot leave the report corrupted or half-written.

The Whole Course, as One Thread

Unit 1 — OS Foundations & Structure

The editor process exists and acts only through system calls into the kernel



Unit 2 — Process Management & Threads

A background autosave thread shares the editor's address space



Unit 3 — CPU Scheduling

The scheduler interleaves the editor with every other running program



Unit 4 — Synchronization

A lock keeps the main and autosave threads from racing on the shared buffer



Unit 5 — Deadlocks

Consistent lock ordering keeps two cooperating threads from freezing each other permanently



Unit 6 — Memory Management

The report's pages are demand-paged in, and shared copy-on-write across windows



Unit 7 — Storage & File Systems

The save itself passes through the file-system layers and lands, journaled or copy-on-write, on disk

Where This Leads Next

This course deliberately stayed within a single machine, running a single instance of one operating system. Several natural directions extend everything you've learned here.



Distributed Operating Systems

What happens when processes on entirely different machines need to synchronize, share files, or survive one machine's outright failure — needing its own versions of synchronization and deadlock detection, across a network that can partially fail



Virtualization and Containers

Running multiple, isolated “guest” operating systems (or lightweight process groups) on one physical machine, depending directly on this course's memory-management and protection material, pushed one level further



Security and Advanced Protection

This course's owner/group/other and ACL models are the starting point for capability-based systems, sandboxing, and formal models of what a compromised process can and cannot do



Real-Time Operating Systems

Environments where a scheduling decision isn't about fairness or throughput, but about hard deadlines that must never be missed — a fundamentally different take on this course's CPU-scheduling unit

Course Complete

You started this course asking what an operating system actually is, and built a complete answer: how it structures itself, manages processes and threads, decides what runs when, keeps concurrent access safe and deadlock-free, gives every process the illusion of abundant memory, and finally turns that memory into data that genuinely survives on disk.

That habit of mind — is this process isolated correctly, is this resource synchronized safely, is memory used efficiently, and is every write to disk actually going to survive — is what this course was built to give you. Congratulations on completing Operating Systems (CSC323).